CUDA SFM LM progress

Timing breakdown and a general-solver baseline

Dataset: Dubrovnik 135 | dense Schur | 3 LM iterations

0.426 s
Current graph API
mean of 3 timed runs

0.138 s
CUDA backend
specialized dense-Schur path

Hybrid
Baseline question
CPU nonlinear work + GPU solve

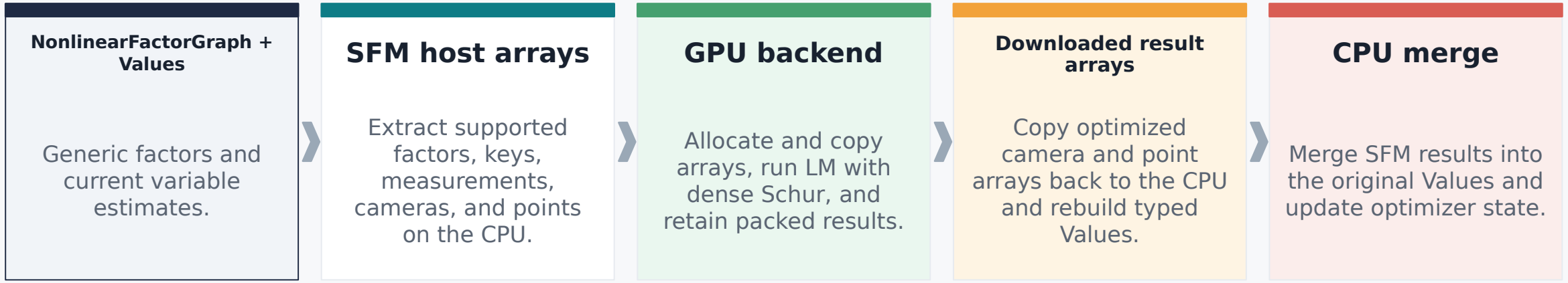
This update has two parts: where the current API path spends time, then whether a general CPU/GPU separation is plausible.

1. **Profile current path**
   Measured
2. **Separate general baseline**
   Proposed
3. **Measure full LM convergence**
   Next

Current implementation

Measured

The public graph API adapts generic GTSAM objects into a specialized CUDA SFM backend.



What the timing includes

The measured API includes CPU graph conversion, GPU setup and solving, and reconstruction of the final GTSAM Values.

Whole graph API: 0.426 s

Measured

Measured scope: warm-up excluded, mean of 3 runs.

426.223 ms

Graph API total

optimizer + optimize + result/error

The API denominator includes construction and final result/error queries, not just optimize().

- Optimizer construction 36.203 ms (8.49%)
- optimize() 376.192 ms (88.26%)
- Result/error queries 13.828 ms (3.24%)

Component	Time	% of graph API
Optimizer construction	36.203 ms	8.49%
optimize()	376.192 ms	88.26%
Result/error queries	13.828 ms	3.24%

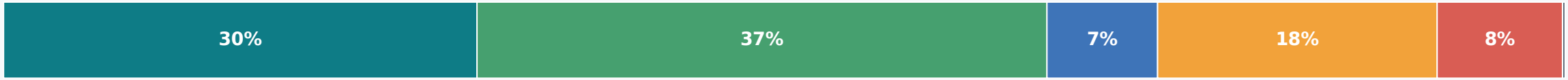
Scope reminder

The following slides unpack the 376.192 ms optimize() call.

Inside optimize(): 0.376 s

Measured

Measured components. Stacked bar: % of optimize(); table: % of Graph API.



- Graph conversion 114.246 ms
- CUDA backend 137.542 ms
- Backend return / assignment 26.685 ms
- Values merge / state update 67.464 ms
- Converted-data destruction 30.247 ms
- Residual 0.008 ms

Measured component	Time	% of whole API
Graph conversion	114.246 ms	26.80%
CUDA backend	137.542 ms	32.27%
Backend return / assignment	26.685 ms	6.26%
Values merge / state update	67.464 ms	15.83%
Converted-data destruction	30.247 ms	7.10%
Residual	0.008 ms	0.00%

CPU result management

97.711 ms

Values merge plus temporary converted-data destruction

Graph conversion and CPU result-management are prominent costs beside the backend.

Inside the CUDA backend: 0.138 s

Measured

Measured hierarchy within the specialized backend.

Backend layer	Time	% of backend
Setup	67.544 ms	49.11%
Solve loop	29.968 ms	21.79%
Download Values	40.030 ms	29.10%

Setup: projection host build 38.686 ms

The largest setup detail is CPU construction of the projection batch.

Solve loop: 29.968 ms

Dense Schur 23.009 ms | Hessian / damping diagonal 2.979 ms | linearized error change 3.655 ms

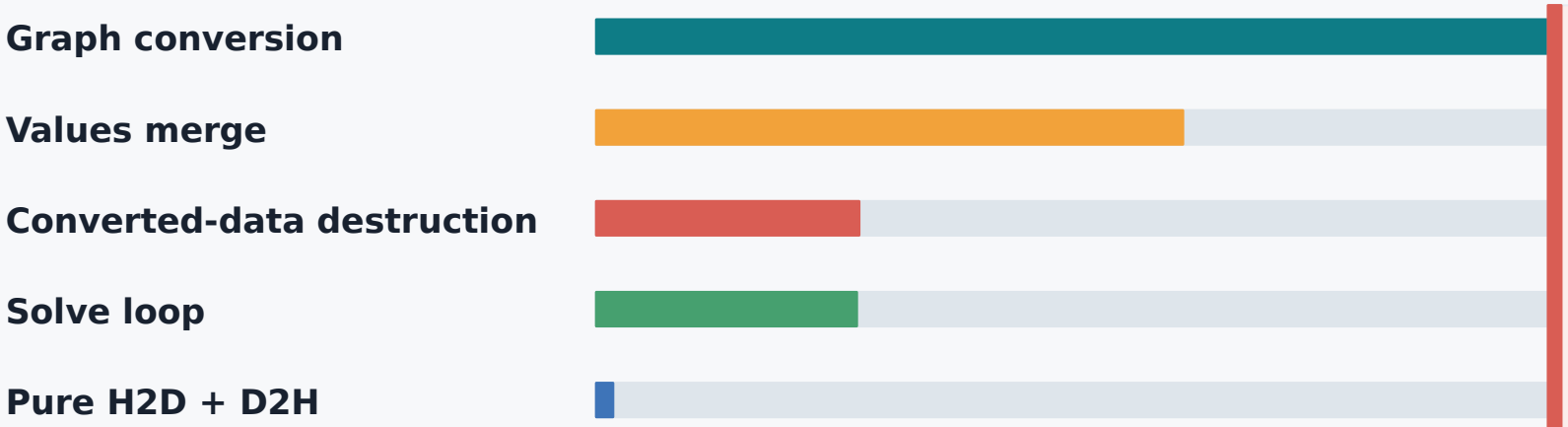
Download Values: 40.030 ms

Raw D2H 0.615 ms | Values rebuild 15.537 ms | remaining host allocation, wrapper, destruction, and tail 23.878 ms

What the profile says

Measured

Measured specialized path: representation conversion and state management dominate the current bottleneck.



Measured component	Time	% of graph API
Graph conversion	114.246 ms	26.80%
Values merge	67.464 ms	15.83%
Converted-data destruction	30.247 ms	7.10%
Solve loop	29.968 ms	7.03%
Pure H2D + D2H	1.936 ms	0.45%

Core conclusion

The present bottleneck is CPU-side representation conversion and object/state management, not PCIe copies or the dense-Schur solve.

This conclusion applies to the specialized representation measured here.

General-solver baseline

Proposed

Proposed Ceres-style separation, not an exact description of every Ceres path.

CPU / TBB factor evaluation + linearization

Arbitrary GTSAM factors stay on CPU.

Reusable sparse numeric J / H buffers

Cache structure; refresh numeric values.

H2D

Upload generic linear-system numeric data.

GPU linear solve

Normal equations plus cuDSS or dense solver.

D2H delta

Return only the solved update.

CPU Values retract + trial decision

Manifold retract, error, and trust-region decision.

Why this is general

Arbitrary GTSAM factors remain in the CPU nonlinear layer. The GPU receives a generic linear system rather than an SFM-specific graph representation.

What remains a hypothesis

Sparse packing, repeated numeric upload, delta mapping, and end-to-end convergence must be measured in the actual prototype.

Sources (official Ceres documentation)

Ceres installation | ceres-solver.readthedocs.io/latest/installation.html

Ceres nonlinear least-squares | ceres-solver.readthedocs.io/latest/nnls_solving.html

Baseline feasibility timing

Measured | Estimated

Separately timed, synthetic per-candidate lower-bound arithmetic. Repeat-0-excluded sensitivity view.

MEASURED CPU OBSERVATIONS

CPU operation	Mean	What was timed
CPU / TBB linearize	28.316 ms	Factor evaluation and linearization
Values retract	14.332 ms	Apply a prebuilt compatible delta
Trial graph.error	11.440 ms	Evaluate trial state

DERIVED COMPONENT

Synthetic GPU component

8.051 ms

Hessian diagonal + dense Schur combined - projection

DERIVED, NOT END-TO-END | arithmetic combines separately timed components

62.139 ms

DERIVED lower bound

54.088 ms CPU + 8.051 ms GPU

+11.283 ms

ESTIMATED generic transfer

115.1 MB / 9.50 GiB/s

73.422 ms

DERIVED + estimate

still not end-to-end

Caveat: not end-to-end. Sparse packing/layout, repeated upload, delta mapping, and convergence remain unmeasured. Do not compare these per-candidate synthetic figures directly to the 426.223 ms three-iteration API total.

Current assessment and next work

Assessment | Proposed

Measured evidence justifies a hybrid baseline experiment, not a performance claim.

Current assessment

Separately timed CPU nonlinear subtotal is tens of milliseconds per candidate versus the prior 0.426 s three-iteration API run. Scopes differ, so this supports feasibility, not a speedup claim. Expected benefit: generality and removal of specialized graph conversion.

Measured conclusion

Next work

TEAL: ARCHITECTURE PATH

- **Cache sparse structure.**
- **Direct TBB numeric fill.**
- **Upload only numeric values.**

AMBER: VALIDATION & EVALUATION

- **GPU normal equations + cuDSS/dense solver.**
- **Download delta, then CPU retract/error.**
- **Measure full LM convergence.**

Success criterion: a measured end-to-end LM comparison with the same problem, convergence policy, and accounting scope.

Appendix: full dense-Schur breakdown

Measured

Measured mean, Dubrovnik 135, warm-up excluded, 3 runs. Times in milliseconds.

Level	Component	Time	% graph API	% parent
Graph API	Optimizer construction	36.203	8.49%	-
Graph API	optimize()	376.192	88.26%	-
Graph API	Result/error queries	13.828	3.24%	-
optimize	Graph conversion	114.246	26.80%	30.37%
optimize	CUDA backend	137.542	32.27%	36.56%
optimize	Backend return / assignment	26.685	6.26%	7.09%
optimize	Values merge / state update	67.464	15.83%	17.93%
optimize	Converted-data destruction	30.247	7.10%	8.04%
backend	Setup	67.544	15.85%	49.11%
backend	Solve loop	29.968	7.03%	21.79%
backend	Download Values	40.030	9.39%	29.10%

Detail	Time	Detail	Time
Setup: projection host build	38.686 ms	Solve: dense Schur	23.009 ms
Setup: pack values	13.688 ms	Solve: damping diagonal	2.979 ms
Setup: allocate trial values	11.769 ms	Solve: linearized error change	3.655 ms
Transfer: total H2D memcpy	1.526 ms	Transfer: total D2H memcpy	0.615 ms
Download: Values rebuild	15.537 ms	Pure H2D + D2H	2.141 ms

Note: the 1.936 ms specialized pure-copy timing is from a separate transfer run; this appendix shows the dense-Schur transfer rows.

Appendix: feasibility ranges and caveats

Measured | Estimated

Measured operations are separate loops. Derived hybrid values are sensitivity views, not sequential candidate calls.

Operation	All 30 mean [min, max] / CV	Repeat-0-excluded 27 mean [min, max] / CV
CPU / TBB linearize	30.851 [25.119, 57.299] / 26.39%	28.316 [25.119, 38.918] / 9.73%
Values retract	14.260 [12.320, 18.152] / 8.79%	14.332 [12.591, 18.152] / 8.71%
Trial graph.error	11.644 [8.518, 16.235] / 20.00%	11.440 [8.518, 14.845] / 19.26%
GPU projection	0.384 [0.379, 0.391] / 0.79%	0.384 [0.379, 0.391] / 0.82%
GPU Hessian diagonal	0.919 [0.909, 0.924] / 0.39%	0.919 [0.909, 0.924] / 0.41%
GPU dense Schur combined	7.513 [7.468, 7.576] / 0.42%	7.516 [7.468, 7.576] / 0.43%

First-index sensitivity

Index-aligned CPU subtotal: repeat 0 mean 80.752 ms, repeat 1 mean 56.881 ms, repeat 2 mean 57.024 ms. Repeat 0 is materially slower.

Transfer formula + provenance

553,336 x (24 Jacobian + 2 residual) x 8 B = 115,093,888 B = 115.1 MB. / 9.50 GiB/s = 11.283 ms.
Dataset: dubrovnik-135-90642-pre.txt; AMD EPYC Milan; NVIDIA A100 80 GB PCIe. CUDA, driver, and TBB thread configuration not recorded.

Scope caveat: independent CPU/GPU loops; synthetic arithmetic excludes sparse packing/layout, repeated upload, delta download/mapping, and LM convergence. Lower bounds: all samples 64.803 ms; repeat-0-excluded 62.139 ms; with estimated transfer 73.422 ms.